

# NAVISAFE



**Autors: Miquel Calzada, Guillem Tarradas**

**Curs: Sistemes Microinformàtics i Xarxes**

**Centre: Institut de Palamós**

**Data: 07/04/2026**

# ÍNDEX

|                                               |           |
|-----------------------------------------------|-----------|
| <b>1. Resum executiu</b>                      | <b>3</b>  |
| 2. Context i situació inicial                 | 4         |
| <b>3. Abast del projecte</b>                  | <b>5</b>  |
| <b>Funcionalitats incloses</b>                | <b>5</b>  |
| <b>Limitacions</b>                            | <b>5</b>  |
| <b>4. Casos d'ús</b>                          | <b>6</b>  |
| CU-01: Monitoratge del vaixell                | 6         |
| CU-02: Planificació de rutes                  | 8         |
| CU-03: Dades de la navegació                  | 10        |
| CU-04: Revisió prèvia                         | 12        |
| CU-05: Gestió de rutes al cloud               | 14        |
| <b>5. Disseny i solució tècnica</b>           | <b>16</b> |
| <b>5.1 Descomposició en capes</b>             | <b>16</b> |
| CU-01: Monitoratge del Vaixell                | 16        |
| CU-02: Dades de la navegació                  | 17        |
| CU-03: Planificació de rutes                  | 18        |
| 5.2 Arquitectura del sistema                  | 19        |
| 5.3 Nucli del sistema                         | 19        |
| 5.4 Flux de dades                             | 19        |
| 5.5 Tecnologies i eines triades               | 19        |
| <b>6. Planificació del projecte</b>           | <b>20</b> |
| 6.1 Fases del projecte                        | 20        |
| 01 Fase inicial                               | 20        |
| 02 Fase de planificació                       | 20        |
| 03 Fase desenvolupament                       | 20        |
| 04 Fase de tancament                          | 20        |
| 7. Obtenció de les tasques                    | 21        |
| 7.1 taula de les tasques per cas d'ús i capa  | 21        |
| 7.2 Tasques principals                        | 22        |
| T-01 Instal·lació de l'entorn base            | 22        |
| T-02 Implementació dels nodes ROS2            | 22        |
| T-03 Backend PHP i base de dades              | 22        |
| T-04 Landing page pública                     | 23        |
| T-05 Validació i documentació final           | 23        |
| <b>7.3 diagrama de gant</b>                   | <b>24</b> |
| <b>8. Desenvolupament i implementació</b>     | <b>25</b> |
| 8.1 Metodologia de treball                    | 25        |
| <b>8.2 Desenvolupament de les pàgines web</b> | <b>25</b> |
| 8.3 Nodes ROS2                                | 25        |
| 8.4 Bridge                                    | 26        |

|                                        |           |
|----------------------------------------|-----------|
| 8.5 Backend PHP                        | 27        |
| 8.6 Estructura del repositori          | 28        |
| <b>9. Desplegament</b>                 | <b>29</b> |
| 9.1 Entorn de producció                | 29        |
| 9.2 Passos de desplegament             | 29        |
| 9.3 Incidències trobades i solucions   | 30        |
| 9.4 Validació final del desplegament   | 31        |
| <b>10. Resultats i validació</b>       | <b>32</b> |
| 10.1 Objectius aconseguits             | 32        |
| 10.2 Proves realitzades                | 33        |
| 10.3 Evidències de funcionament        | 33        |
| 10.4 Feedback rebut                    | 33        |
| <b>11. Conclusions i aprenentatges</b> | <b>34</b> |
| 11.1 Aprenentatges tècnics             | 34        |
| 11.3 Dificultats trobades              | 35        |
| <b>12. Annexos</b>                     | <b>36</b> |
| 12.1 Fragment de codi                  | 36        |
| 12.2 Coneixements del cicle:           | 39        |
| <b>12.3 Wiki</b>                       | <b>40</b> |
| Topics:                                | 40        |
| 1. Hardware Setup & Safety             | 40        |
| 10. Web Apps & CMS                     | 41        |
| 15. Ethical Hacking & Security         | 43        |

# 1. Resum executiu

## Català

El projecte Navisafe neix per millorar la seguretat i el control de petites embarcacions que no disposen de tecnologia avançada. Actualment, moltes d'aquestes barques tenen dificultats per mantenir el rumb, controlar la velocitat o detectar obstacles, sobretot en condicions meteorològiques adverses. Aquest fet pot provocar situacions de risc durant la navegació. Per solucionar aquest problema, a Navisafe proposem un sistema de navegació assistida econòmic i accessible a part de la facilitat d'implementar en embarcacions petites. Aquest sistema ajuda el capità a mantenir el rumb mitjançant una motorització semiautomatitzada, i també optimitza la velocitat per reduir el consum de combustible i millora l'eficiència de la navegació. A més, incorpora navegació per instruments amb mapes digitals i informació d'altres vaixells.

Una altra funcionalitat important és l'estabilització automàtica, que compensa els efectes del vent i les onades per fer la navegació més segura. També inclou una càmera tèrmica i un sonar per detectar obstacles i controlar el fons marí, també oferim càmeres de vigilància al vaixell, les quals pots mirar a través de l'aplicació al teu mòbil desde tot arreu. Tot el sistema està gestionat per un hub central basat en una Raspberry Pi, que permet controlar i visualitzar les dades a través d'una interfície web senzilla. El projecte no inclou la creació de hardware propi ni la instal·lació en embarcacions grans o sistemes professionals.

## Anglès

The Navisafe project was created to improve the safety and control of small boats that do not have advanced technology. At the moment, many of these vessels have difficulties keeping their course, controlling their speed or detecting obstacles, especially in bad weather conditions. This can lead to dangerous situations while sailing. To solve this problem, Navisafe proposes an affordable and accessible assisted navigation system that is also easy to install on small boats. This system helps the captain to maintain the course through a semi-automatic motor control, and it also optimises the speed to reduce fuel consumption and improve navigation efficiency. In addition, it includes instrument-based navigation with digital maps and information from other vessels. Another important feature is automatic stabilisation, which compensates for the effects of wind and waves to make sailing safer. The system also includes a thermal camera and a sonar to detect obstacles and monitor the seabed. Furthermore, it offers surveillance cameras on the boat, which can be viewed through a mobile application from anywhere. The whole system is managed by a central hub based on a Raspberry Pi, which allows the user to control and view the data through a simple web interface. The project does not include the creation of custom hardware or installation on large vessels or professional systems.



## 2. Context i situació inicial

Moltes persones naveguen amb petites embarcacions que no disposen d'un sistema avançat de navegació, fet que pot dificultar el manteniment del rumb, el control de la velocitat o la detecció d'obstacles en condicions adverses. Això pot augmentar el risc d'errors i situacions perilloses durant la navegació, ja que depenen principalment de sistemes bàsics i de l'experiència del capità.

Per aquest motiu, el nostre projecte Navisafe té com a objectiu desenvolupar un sistema de navegació assistida que millori la seguretat, l'estabilitat i el control d'aquestes embarcacions d'una manera econòmica i accessible.

# 3. Abast del projecte

## Funcionalitats incloses

- Motorització semiautomatitzada per mantenir el rumb seguint una ruta definida pel capità.
- Control de velocitat i optimització del consum per millorar l'eficiència del combustible.
- Navegació assistida per instruments, amb integració de mapes digitals i dades de posicionament.
- Planificació i gestió de rutes, amb ajust automàtic segons condicions meteorològiques.
- Sistema d'estabilització automàtica per compensar l'efecte del vent i les onades.
- Detecció d'obstacles, mitjançant càmeres i sonar per millorar la seguretat.
- Monitoratge en temps real del vaixell, incloent sensors de motor, combustible, bateria i sentina.
- Interfície web centralitzada (dashboard) per al control i visualització del sistema.
- Gestió de rutes al núvol, amb possibilitat de guardar, sincronitzar i reutilitzar trajectes.

## Limitacions

- No es desenvolupa un hardware propi; s'utilitzen sensors i dispositius ja existents.
- No s'implementa en embarcacions grans (superiors a 15 metres d'eslora).
- No contempla certificacions oficials ni ús en entorns comercials regulats.
- No es realitza una integració real en vaixells reals, sinó un prototip funcional o simulació.

## 4. Casos d'ús

### CU-01: Monitoratge del vaixell

**Actor:** Propietari del vaixell

**Funcionalitat:** Supervisió remota de consumibles, motor i càmeres, amb alertes automàtiques i accés via navegador.

**Precondicions:**

- L'usuari ha iniciat sessió amb credencials vàlides.
- El sistema de sensors i Raspberry Pi està operatiu.
- La xarxa Wi-Fi / Ethernet del vaixell funciona.

**Flux principal:**

1. Accés al sistema.
2. Visualització de combustible, bateries i estat bomba centina
3. Visualització paràmetres del motor.
4. Visualització de càmeres embarcades.
5. Alertes automàtiques per incidències.

**Fluxos alternatius:**

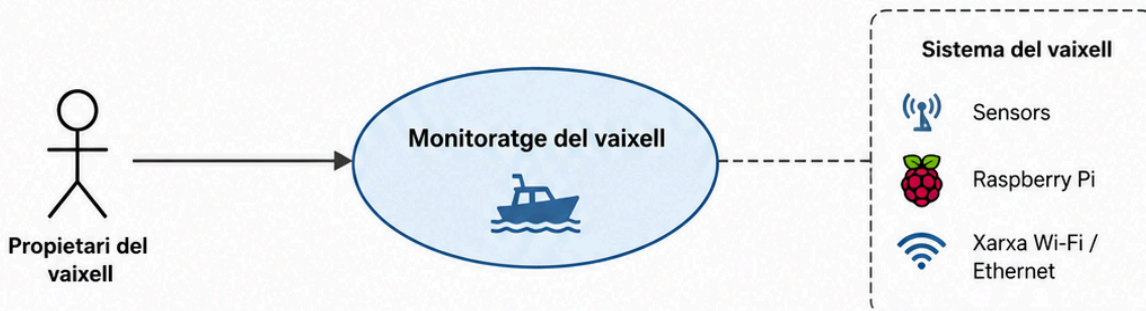
- Sensor fora de servei: si no es reben dades en 2 segons es considera timeout. A partir de 3 errors consecutius s'envia un avís i a 4 es dispara una alarma.
- Sensor d'aigua a la sentina: si es detecta nivell alt d'aigua s'activa l'alarma immediatament.
- Connexió interrompuda: intent de reconexió automàtic i alerta si no es recupera.

**Postcondicions:** Dades i alertes registrades amb traçabilitat.

**Requisits especials:**

- Seguretat: Autenticació i permisos
- Integritat: Control de versions i logs d'auditoria.

## CU-01: Monitoratge del vaixell



**Funcionalitat:** Supervisió remota de consumibles, motor i càmeres, amb alertes automàtiques i accés via navegador.



### Precondicions:

- L'usuari ha iniciat sessió amb credencials vàlides.
- El sistema de sensors i Raspberry Pi està operatiu.
- La xarxa Wi-Fi / Ethernet del vaixell funciona.



### Flux principal:

- 1  Accés al sistema.
- 2  Visualització de combustible, bateries i estat bomba cetina.
- 3  Visualització de paràmetres del motor.
- 4  Visualització de càmeres embarcades.
- 5  Alertes automàtiques per incidències.



### Fluxos alternatius:



#### Sensor fora de servei:

- si no es reben dades en 2 segons es considera timeout.
- A partir de 3 errors consecutius s'envia un avís i a 4 es dispara una alarma.
- Es pot notificar per correu i permetre consultar càmera o temperatura.



#### Sensor d'aigua a la sentina:

- si es detecta nivell alt d'aigua s'activa l'alarma immediatament.
- Es recomana fer un test abans de navegar o de manera setmanal per comprovar el correcte funcionament del sistema.



#### Connexió interrompuda:

- si es perd la connexió es fa un intent automàtic de reconexió i, si no es recupera, s'envia una alerta.



**Postcondicions:** Dades i alertes registrades amb traçabilitat.



### Requisits especials:

-  **Seguretat:** Autenticació i permisos.
-  **Integritat:** Control de versions i logs d'auditoria.

## CU-02: Planificació de rutes

**Actor:** Propietari del vaixell

**Funcionalitat:** Planificació de rutes amb ajust automàtic segons condicions meteorològiques.

### Precondicions:

- Sistema GPS i sensors meteorològics operatius.
- Sistema de control de rutes en funcionament.
- Xarxa i Raspberry Pi actius.

### Flux principal:

1. L'usuari prepara la ruta amb punts de parada i trajectes.
2. El sistema ajusta la ruta segons vent, onatge i corrents.
3. Es carrega la ruta al sistema del vaixell.
4. Es recalcula automàticament si canvien les condicions.
5. Visualitza rutes i alertes a la interfície web.

### Fluxos alternatius:

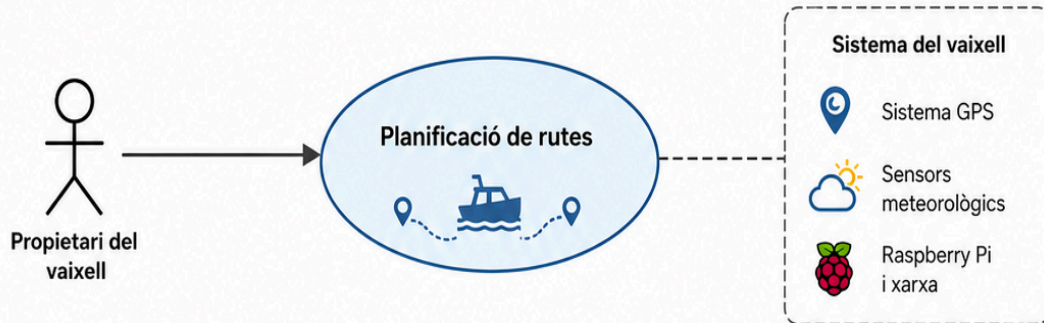
- Predicció meteorològica no disponible → ruta mantinguda sense ajustos.
- GPS fora de servei → recalcul automàtic bloquejat i alerta.
- rutes fetes anteriorment i adaptarem a cdd

**Postcondicions:** Ruta optimitzada i segura.

### Requisits especials:

- Seguretat: Accés només a usuaris autoritzats.
- Disponibilitat: Actualització de rutes en temps real.
- Integritat: Historial de rutes amb versions i alertes documentades.

## CU-02: Planificació de rutes



**Funcionalitat:** Planificació de rutes amb ajust automàtic segons condicions meteorològiques.



### Precondicions:

- Sistema GPS i sensors meteorològics operatius.
- Sistema de control de rutes en funcionament.
- Xarxa i Raspberry Pi actius.



### Flux principal:

- L'usuari prepara la ruta amb punts de parada i trajectes.
- El sistema ajusta la ruta segons vent, onatge i corrents.
- Es carrega la ruta al sistema del vaixell.
- Es recalcula automàticament si canvien les condicions.
- Visualitza rutes i alertes a la interfície web.



### Fluxos alternatius:



**Predicció meteorològica no disponible**  
→ ruta mantinguda sense ajustos.



**GPS fora de servei**  
→ recalcul automàtic bloquejat i alerta.



**Rutes fetes anteriorment i adaptarem a CDD**  
→ es poden reutilitzar i adaptar segons condicions actuals.



**Postcondicions:** Ruta optimitzada i segura.



### Requisits especials:

- Seguretat:** Accés només a usuaris autoritzats.
- Disponibilitat:** Actualització de rutes en temps real.
- Integritat:** Historial de rutes amb versions i alertes documentades.

## CU-03: Dades de la navegació

**Actor:** Propietari vaixell

**Funcionalitat:** Selecció de paràmetres i generació de gràfics/informes.

**Precondicions:**

- Sistema de sensors operatiu.
- disponibles: ruta GPS, velocitat i consum.
- Interfície web en funcionament.

**Flux principal:**

1. Visualització del panell del vaixell amb:
  - Estat del vaixell consum i combustible disponible.
  - Navegació amb mapa i carta nàutica.
2. Visualització de dades en temps real al navegador.
3. Genera gràfics i informes.
4. Accedeix a les dades tant a bord com remotament.
5. Les dades es queden registrades per anàlisi futura.

**Fluxos alternatius:**

- Sensor no disponible → notificació d'error
- Error en generació de gràfics → regeneració o exportació de dades.

**Postcondicions:** Historial de dades disponible per anàlisi futura.

**Requisits especials:**

- Rendiment: Generació de gràfics sense retard perceptible.
- Integritat: Dades sense pèrdua ni corrupció.



## CU-03: Dades de la navegació



**Funcionalitat:** Selecció de paràmetres i generació de gràfics/informes.



### Precondicions:

- Sistema de sensors operatiu.
- Dades disponibles: ruta GPS, velocitat i consum.
- Interfície web en funcionament.



### Flux principal:

Visualització del pànell del vaixell amb:

1



Estat del vaixell, consum i combustible disponible



68%



Navegació amb mapa i carta nàutica



2



Visualització de dades en temps real al navegador.

3



Genera gràfics i informes.

4



Accedeix a les dades tant a bord com remotament.

5



Les dades es queden registrades per anàlisi futura.



### Fluxos alternatius:



Sensor no disponible  
→ notificació d'error



Error en generació de gràfics  
→ regeneració o exportació de dades.



**Postcondicions:** Historial de dades disponible per anàlisi futura.



### Requisits especials:

- **Rendiment:** Generació de gràfics sense retard perceptible.
- **Integritat:** Dades sense pèrdua ni corrupció.



## CU-04: Revisió prèvia

**Actor:** Propietari del vaixell

**Funcionalitat:** Verificació del sistema abans de navegar mitjançant el navegador, assegurant que tots els sensors, dispositius i components crítics funcionen correctament.

### Precondicions:

- L'usuari ha iniciat sessió amb credencials vàlides.
- El sistema Raspberry Pi està operatiu.
- Connexió de xarxa disponible (Wi-Fi o Ethernet).
- Sensors i dispositius connectats al sistema.

### Flux principal:

1. L'usuari accedeix al sistema des del navegador.
2. Selecciona l'opció de "test abans de sortir".
3. El sistema executa una comprovació automàtica dels sensors (combustible, bateries, temperatura, bomba de sentina, etc.).
4. Es verifica el funcionament de la bomba de sentina mitjançant un test d'activació.
5. Es comprova l'estat del motor i altres components crítics.
6. Es validen les càmeres embarcades i la seva visualització.
7. Es verifica el timó
8. El sistema mostra un informe amb l'estat de tots els elements (correcte / error).

### Fluxos alternatius:

- Sensor no respon → notificació després de timeout de 2 segons.
- 3 errors consecutius → alerta al sistema.
- 4 errors consecutius → enviament de notificació (correu electrònic o alerta)
- Error en la bomba de sentina → alerta crítica de risc (possible acumulació d'aigua Càmera no disponible → notificació i opció de reintent.
- Connexió interrompuda → intent de reconexió automàtic i alerta.

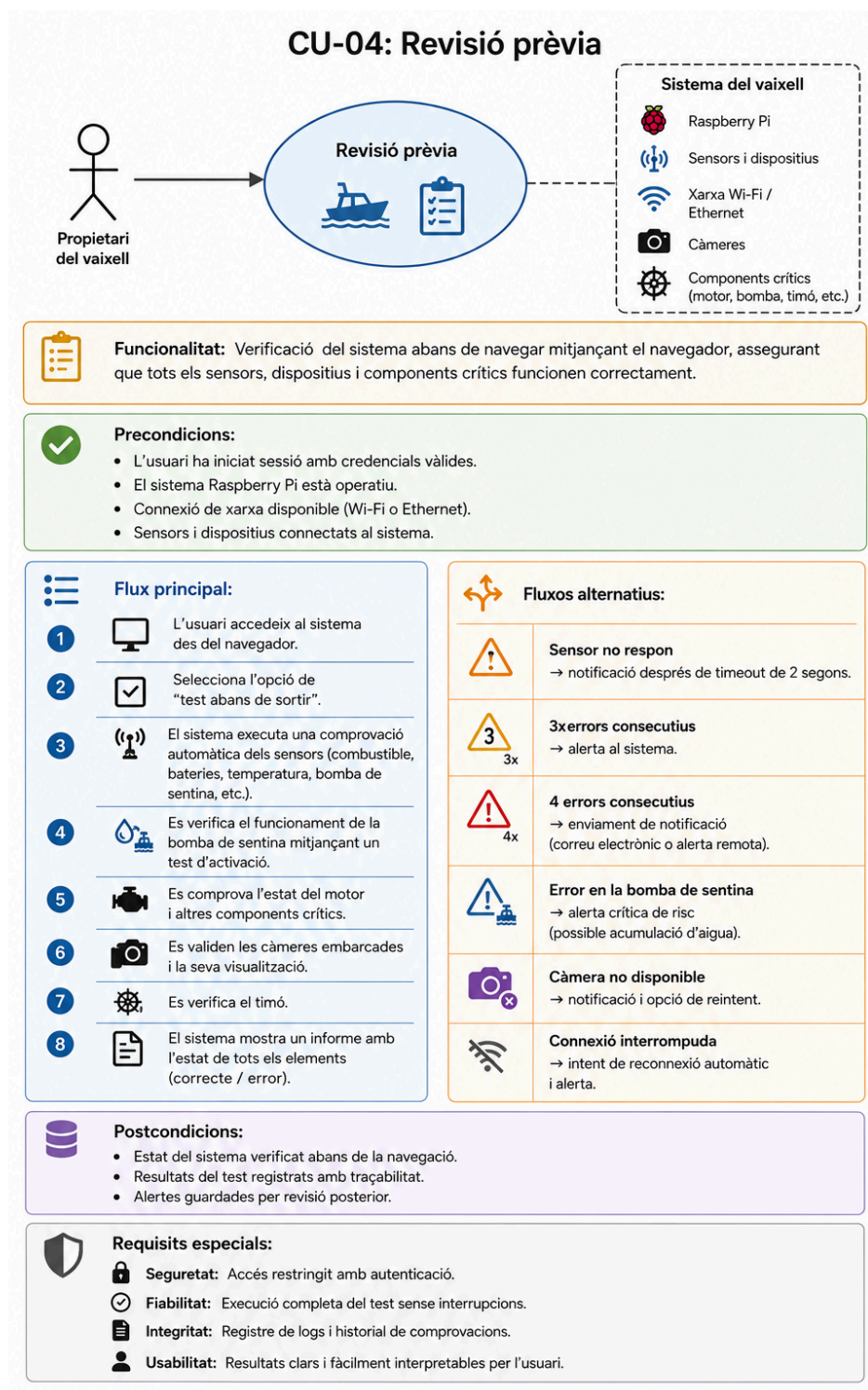
### Postcondicions:

- Estat del sistema verificat abans de la navegació.
- Resultats del test registrats amb traçabilitat.
- Alertes guardades per revisió posterior.

### Requisits especials:

- **Seguretat:** Accés restringit amb autenticació.

- **Fiabilitat:** Execució completa del test sense interrupcions.
- **Integritat:** Registre de logs i historial de comprovacions.
- **Usabilitat:** Resultats clars i fàcilment interpretables per l'usuari.



## CU-05: Gestió de rutes al cloud

**Actor:** Propietari del vaixell

**Funcionalitat:** Pujada, emmagatzematge i descàrrega de rutes al cloud per a la seva reutilització i sincronització amb el vaixell.

### Precondicions:

- L'usuari ha iniciat sessió amb credencials vàlides.
- Connexió activa
- Sistema de rutes i GPS operatiu
- Servei cloud disponible

### Flux principal:

1. L'usuari crea o selecciona una ruta de navegació.
2. Puja la ruta al cloud des de la interfície web.
3. El sistema valida i guarda la ruta al servidor.
4. L'usuari pot accedir a les rutes guardades des de qualsevol dispositiu.
5. L'usuari selecciona una ruta del cloud.
6. La ruta es descarrega automàticament al sistema del vaixell.
7. La ruta queda disponible per a navegació directa o planificació.

### Fluxos alternatius:

- Error de connexió amb el cloud → reintent automàtic i notificació.
- Ruta corrupta o incompleta → rebuig de pujada amb avís al usuari.
- Conflicte de versions de ruta → el sistema manté historial i permet escollir versió.

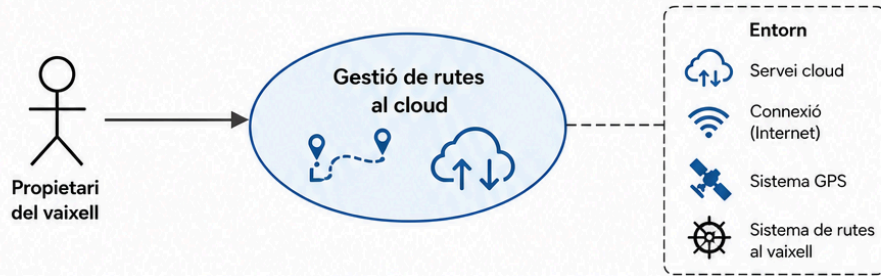
### Postcondicions:

- Ruta emmagatzemada correctament al cloud.
- Ruta sincronitzada i disponible al vaixell.
- Historial de versions de rutes guardat.

### Requisits especials:

- **Seguretat:** Accés al cloud només amb usuari autènticat.
- **Disponibilitat:** Sincronització de rutes en qualsevol moment.
- **Integritat:** Dades de rutes sense pèrdua ni corrupció.
- **Traçabilitat:** Registre de pujades, descàrregues i versions.

## CU-05: Gestió de rutes al cloud



**Funcionalitat:** Pujada, emmagatzematge i descàrrega de rutes al cloud per a la seva reutilització i sincronització amb el vaixell.



### Precondicions:

- L'usuari ha iniciat sessió amb credencials vàlides.
- Connexió activa
- Sistema de rutes i GPS operatiu
- Servei cloud disponible



### Flux principal:

- 1 L'usuari crea o selecciona una ruta de navegació.
- 2 Puja la ruta al cloud des de la interfície web.
- 3 El sistema valida i guarda la ruta al servidor.
- 4 L'usuari pot accedir a les rutes guardades des de qualsevol dispositiu.
- 5 L'usuari selecciona una ruta del cloud.
- 6 La ruta es descarrega automàticament al sistema del vaixell.
- 7 La ruta queda disponible per a navegació directa o planificació.



### Fluxos alternatius:



**Error de connexió amb el cloud**  
→ reintent automàtic i notificació.



**Ruta corrupta o incompleta**  
→ rebuig de pujada amb avis al usuari.



**Conflicte de versions de ruta**  
→ el sistema manté historial i permet escollir versió.



### Postcondicions:

- Ruta emmagatzemada correctament al cloud.
- Ruta sincronitzada i disponible al vaixell.
- Historial de versions de rutes guardat.



### Requisits especials:

- Seguretat:** Accés al cloud només amb usuari autènticat.
- Disponibilitat:** Sincronització de rutes en qualsevol moment.
- Integritat:** Dades de rutes sense pèrdua ni corrupció.
- Traçabilitat:** Registre de pujades, descàrregues i versions.

# 5. Disseny i solució tècnica

## 5.1 Descomposició en capes

### CU-01: Monitoratge del Vaixell

Capa física (hardware):

- Sensors: combustible, temperatura, GPS, AIS, càmera i sensor aigua
- Raspberry Pi
- Pantalla tàctil / tablet

Xarxa i comunicacions:

- Wi-Fi / Ethernet local.
- Firewall

Sistema operatiu i entorns virtuals

- Ubuntu

Serveis:

- ROS2 com a nucli central per recollir i distribuir les dades dels sensors
- Servidor web PHP que mostra les dades en temps real al dashboard i genera alertes
- https

Aplicacions:

- Web Navisafe: pàgina pública informativa
- Dashboard a la Raspberry Pi: visualització de sensors i alertes

Mapa interactiu amb dues opcions:

- Via cloud: rutes pujades i compartides per altres usuaris
- Via web local: rutes pròpies gestionades des del panell de control

## CU-02: Dades de la navegació

Capa física (hardware):

- Sensors: combustible, temperatura, GPS, AIS, càmera
- Raspberry Pi
- Pantalla tàctil / tablet

Xarxa i comunicacions:

- Wi-Fi / Ethernet local.
- Firewall
- Accés remot via Internet
- Comunicació segura

Sistema operatiu i entorns virtuals

- Ubuntu

Serveis:

- ROS 2 com a nucli central per recollir, processar i distribuir les dades dels sensors.
- Servidor web PHP que mostra les dades en temps real al dashboard i genera alertes.
- https
- seleccions el panell que vols i el seu entorn

Aplicacions:

- Visualització de dades en temps real
- Generació de gràfics
- Exportació d'informes
- Historial de dades

## CU-03: Planificació de rutes

Capa física (hardware):

- Sensors simulats: combustible, temperatura, GPS, AIS, càmera i sensors meteorològics
- Raspberry Pi
- Pantalla tàctil / tablet

Xarxa i comunicacions:

- Wi-Fi / Ethernet local
- Firewall
- Accés remot via Internet
- Comunicació segura

Sistema operatiu i entorns virtuals

- Ubuntu

Serveis:

- ROS 2 com a nucli central per recollir, processar i distribuir les dades dels sensors.
- Servidor web PHP que mostra les dades en temps real al dashboard i genera alertes.
- https
- Base de dades per historial de rutes
- Motor de càlcul de rutes

Aplicacions:

- Planificador de rutes
- Visualització del mapa
- Alertes meteorològiques
- Historial de rutes

## 5.2 Arquitectura del sistema

- El sistema s'organitza en cinc capes funcionals:
- **Capa física:** sensors encarregats de captar dades.
- **Capa de comunicacions:** garanteix la connectivitat entre components.
- **Sistema operatiu:** Ubuntu, responsable de la gestió de recursos.
- **Capa de serveis:** processament i emmagatzematge de dades.
- **Capa d'aplicacions:** visualització de la informació per a l'usuari.

## 5.3 Nucli del sistema

- El nucli és una Raspberry Pi 4 que actua com a servidor local.
- Rep dades dels sensors via USB
- Distribueix la informació als usuaris mitjançant la Xarxa local.
- Permet accés remot seguir via VPN o configuracions de xarxa.

## 5.4 Flux de dades

- segueix el següent procés:
- Els sensors capten dades físiques i els envien a ROS2
- Un node ROS2 llegeix les dades i escriu els valors en una base de dades
- El servidor web PHP consulta la base de dades periòdicament i serveix les dades al navegador de l'usuari
- El panell de control interpreta i visualitza en temps real
- Per a la planificació de rutes segueix el següent procés:
- L'usuari introdueix rutes mitjançant la interfície web.
- El sistema processa les rutes
- Es generen ordres per al control del timó.
- Ajust del rumb de manera semiautomàtica segons condicions.

## 5.5 Tecnologies i eines triades

- Raspberry Pi 4
- Ubuntu Server
- ROS2
- Python 3
- PHP, HTML
- HTTP dades en temps real
- Leaflet.js



# 6. Planificació del projecte

## 6.1 Fases del projecte

### 01 Fase inicial

- definició del sistema Navisafe
- Definició de les necessitats i límits que cobreix el sistema

### 02 Fase de planificació

- Definició de cada cas d'ús
- Descomposició en capes
- Creació del diagrama de Gantt

### 03 Fase desenvolupament

- Desenvolupament del panell de control a la web
- Implementació amb ROS2
- Connexió amb sensors simulats
- Creació de base de dades
- Integració de mòduls

### 04 Fase de tancament

- Proves del sistema
- Validació de funcions
- Correcció d'errors
- Preparació de la documentació final

# 7. Obtenció de les tasques

## 7.1 taula de les tasques per cas d'ús i capa

| Cas d'ús / Capa                      | Capa Física                                       | Xarxa i Comunicacions         | Sistema Operatiu         | Serveis                  | Aplicacions                |
|--------------------------------------|---------------------------------------------------|-------------------------------|--------------------------|--------------------------|----------------------------|
| <b>CU-01 Monitoratge del vaixell</b> | Connexió de sensors i actuadors a la Raspberry Pi | Configuració Wi-Fi i Ethernet | Instal·lació Ubuntu      | ROS2 i servidor web      | Dashboard i alertes        |
| <b>CU-02 Planificació de rutes</b>   | Configuració GPS i sensors                        | Accés remot i VPN             | Configuració del sistema | Motor de càlcul de rutes | Mapa interactiu            |
| <b>CU-03 Dades de navegació</b>      | Sensors de consum, temperatura i actuadors        | Comunicació segura HTTPS      | Supervisió de serveis    | Base de dades SQLite     | Gràfics i informes         |
| <b>CU-04 Revisió prèvia</b>          | Test de sensors i actuadors                       | Verificació connexió          | Control de processos     | Servei de verificació    | Panell de revisió          |
| <b>CU-05 Gestió de rutes cloud</b>   | Raspberry Pi i emmagatzematge                     | Connexió Internet i firewall  | Gestió de sessions       | APIs de rutes            | Historial i sincronització |

## 7.2 Tasques principals

### T-01 Instal·lació de l'entorn base

Preparació de la Raspberry Pi amb Ubuntu Server i configuració de la connexió remota.

#### Resultat

Sistema Ubuntu funcional i accessible des de la xarxa.

#### DoD

- Ubuntu instal·lat
- SSH operatiu
- IP estàtica configurada

**fitxa:** Disposar d'un sistema base estable i accessible remotament.

### T-02 Implementació dels nodes ROS2

Creació dels nodes ROS2 per llegir les dades dels sensors.

#### Resultat

Nodes GPS, Fuel, Motor i Sentina funcionals.

#### DoD

- Publicació de dades en temps real
- Detecció d'errors implementada
- Generació de fitxers JSON

**fitxa:** Implementar la captura i transmissió de dades dels sensors amb ROS2.

### T-03 Backend PHP i base de dades

Desenvolupament del sistema de login i emmagatzematge de rutes.

#### Resultat

Sistema d'autenticació i historial de rutes funcional.

## **DoD**

- Login amb sessions
- Base de dades SQLite
- APIs save\_route.php i get\_routes.php

**Fita:** Desenvolupar un backend funcional per gestionar usuaris i rutes.

## **T-04 Landing page pública**

Creació de la pàgina principal informativa de Navisafe.

### **Resultat**

Web responsive amb informació del projecte.

## **DoD**

- Disseny responsive
- Navegació funcional
- Accés al dashboard

**Fita:** Crear una interfície web pública moderna i responsive.

## **T-05 Validació i documentació final**

Realització de proves finals i preparació de la documentació.

### **Resultat**

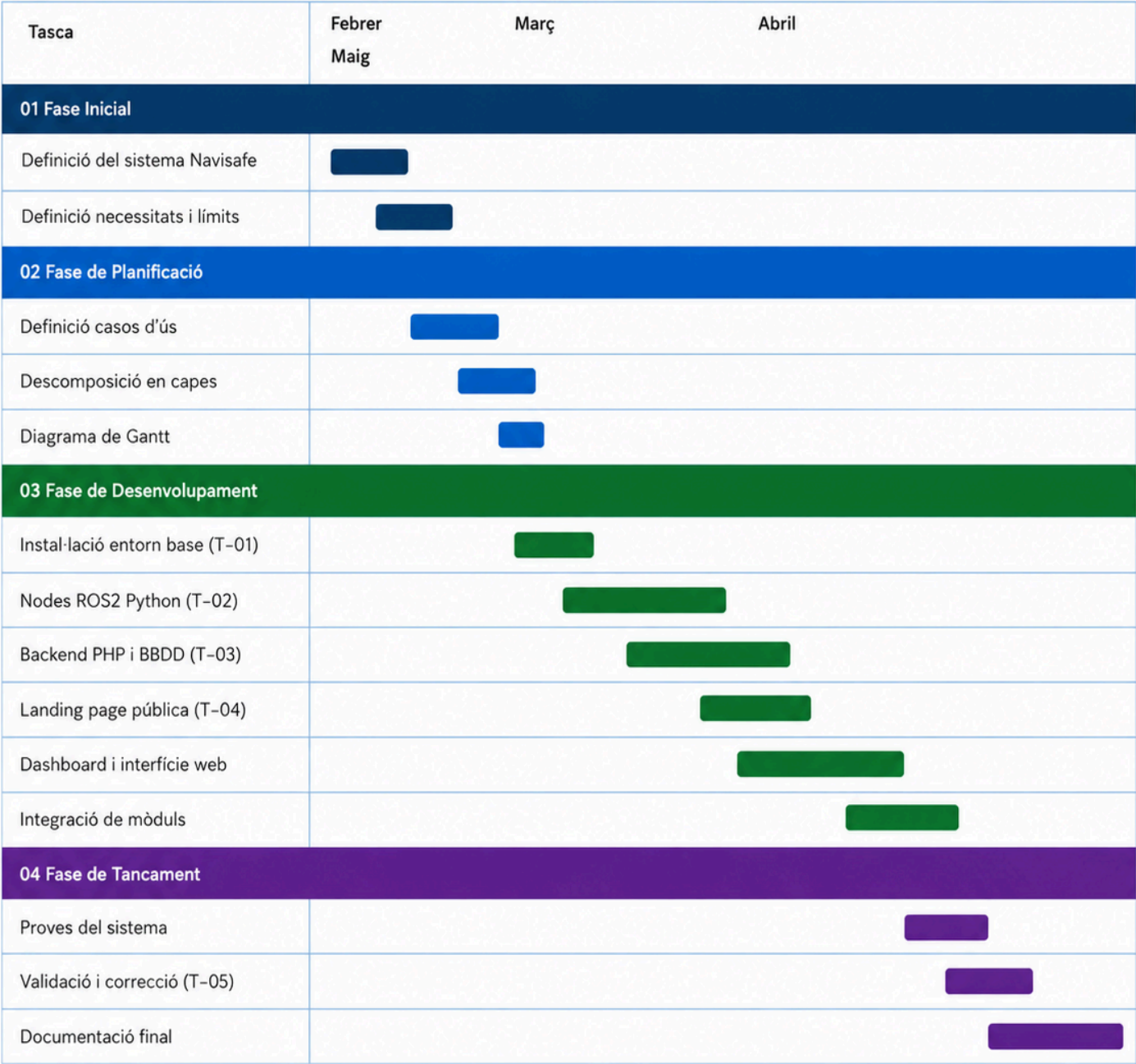
Sistema completament provat i documentat.

## **DoD**

- Proves superades
- Dashboard funcional
- Documentació preparada

**Fita:** Validar el funcionament global del sistema i completar la documentació.

# 7.3 diagrama de gant



| Color       | Fase                       | Període         |
|-------------|----------------------------|-----------------|
| ■ Blau fosc | 01 Fase Inicial            | 9 feb – 20 feb  |
| ■ Blau      | 02 Fase de Planificació    | 18 feb – 10 mar |
| ■ Verd      | 03 Fase de Desenvolupament | 10 mar – 2 mai  |
| ■ Violeta   | 04 Fase de Tancament       | 24 abr – 13 mai |

# 8. Desenvolupament i implementació

## 8.1 Metodologia de treball

El projecte Navisafe s'ha desenvolupat de manera progressiva, començant per les parts bàsiques del sistema i acabant amb la interfície web. Primer es van preparar els sensors i actuadors, després el backend i finalment el dashboard web.

## 8.2 Desenvolupament de les pàgines web

El projecte inclou dues pàgines principals:

| Pàgina                          | Funció                                      |
|---------------------------------|---------------------------------------------|
| index.html                      | Landing page pública                        |
| dashboard.html                  | Panell de control privat                    |
| Backend PHP (login.php, db.php) | Autenticació d'usuaris i gestió de sessions |

Les dues pàgines es van crear amb HTML i CSS bàsic desenvolupat per nosaltres.

Després vam utilitzar IA per millorar:

- El disseny visual
- L'estructura de la web
- L'estil responsive
- La distribució dels elements

## 8.3 Nodes ROS2

Inicialment els nodes ROS2 es van començar en C++, però durant les proves vam trobar problemes de compatibilitat a la Raspberry Pi 4 amb ROS2 Humble. Per solucionar-ho, vam investigar projectes reals que usaven ROS2 en sistemes similars. Ens vam basar en el projecte [Angler](#) — un marc de treball en ROS2 pensat per a robots submarins, especialment vehicles amb manipulació, orientat a recerca i desenvolupament — per entendre com estructurar els nodes de manera simple i funcional. Amb aquesta referència i ajuda d'IA vam convertir tots els nodes a Python.

Les funcions principals dels nodes ROS2 són:

- Simular sensors (combustible, motor, GPS, sentina)
- Generar dades en temps real
- Crear fitxers JSON per al bridge
- Enviar dades al dashboard

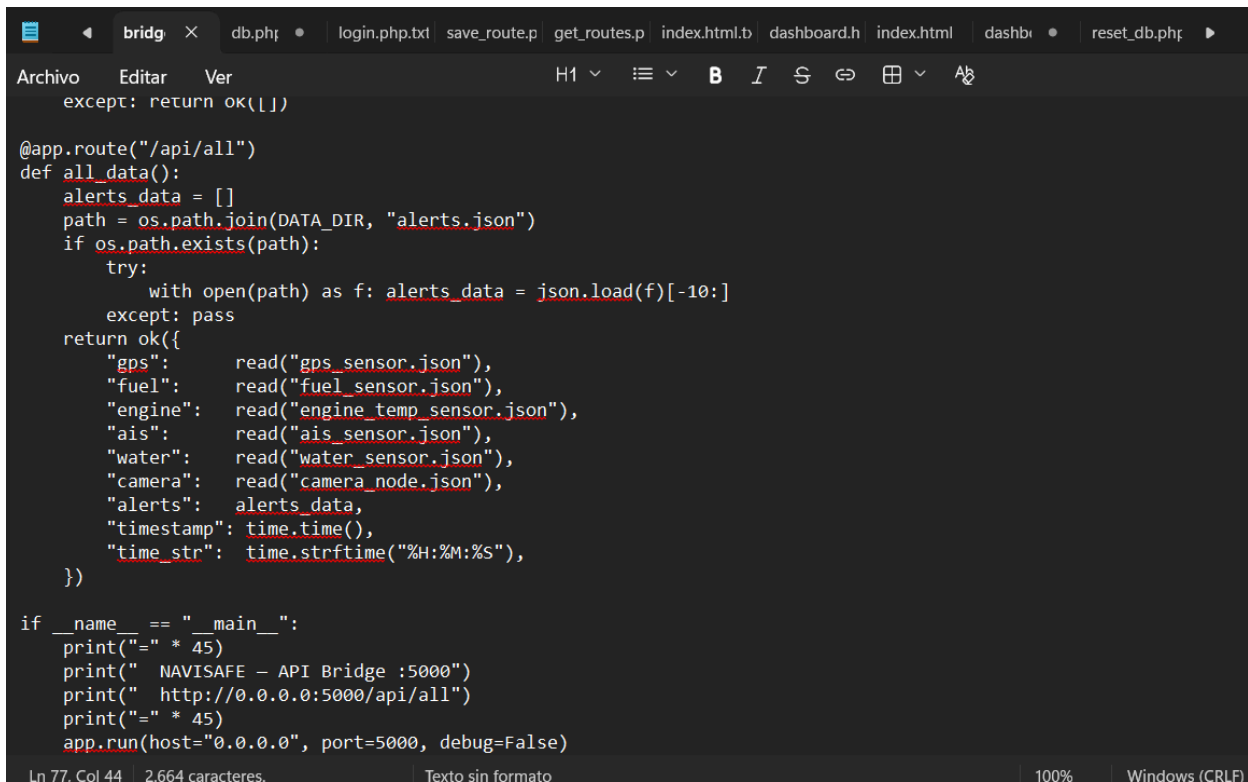
```
39 lon = round(max(MIN_LON, min(MAX_LON, lon)), 6)
42
43 fuel = max(0, round(fuel - random.uniform(0, 0.2), 1))
44 temp = max(60, min(round(temp + random.uniform(-1.5, 2.5), 1), 115))
45 sent = max(0, min(round(sent + random.uniform(-1.0, 2.0), 1), 100))
46 vel = round(random.uniform(0.5, 8.0), 1)
47 rumb = round((angle + 90) % 360)
48 n_ais = random.randint(0, 4)
49 vessels = random.sample(VESSELS, n_ais)
50 cam_ok = random.random() > 0.1
51
52 alertes = []
53 ts = time.strftime("%H:%M:%S")
54 if fuel < 20: alertes.append({"nivell": "WARNING", "msg": f"🔥 Combustible baix: {fuel}%", "hora": ts})
55 if temp > 100: alertes.append({"nivell": "CRITICAL", "msg": f"🔥 Temperatura motor: {temp}°C", "hora": ts})
56 if sent > 25: alertes.append({"nivell": "CRITICAL", "msg": f"💧 Bomba sentina: {sent}%", "hora": ts})
57 if fuel < 10: alertes[0]["nivell"] = "CRITICAL" if alertes else None
58
59 dades = {
60     "gps": {"lat": lat, "lon": lon, "trail": trail, "velocitat": vel, "rumb": rumb},
61     "fuel": {"valor": fuel, "estat": "CRITICAL" if fuel < 10 else "WARNING" if fuel < 20 else "OK"},
62     "motor": {"valor": temp, "estat": "CRITICAL" if temp > 100 else "WARNING" if temp > 90 else "OK"},
63     "sentina": {"valor": sent, "estat": "CRITICAL" if sent > 25 else "WARNING" if sent > 15 else "OK"},
64     "ais": {"vaixells": n_ais, "llista": vessels},
65     "camera": {"ok": cam_ok},
66     "alertes": alertes,
67     "hora": ts,
68 }
69 with open(f"{DATA_DIR}/dades.json", "w") as f:
70     json.dump(dades, f)
71
72 print(f"[{ts}] GPS:{lat},{lon} | Fuel:{fuel}% | Temp:{temp}°C | Sent:{sent}% | AIS:{n_ais} | Cam: {'OK' if cam_ok else 'ERR'}")
73
74 print("Navisafe - Sensors en marxa. Ctrl+C per aturar.")
75 while True:
```

## 8.4 Bridge

El fitxer bridge.py connecta els sensors ROS2 amb el dashboard web.

Les seves funcions principals són:

- Llegir dades dels sensors
- Exposar una API REST
- Enviar dades al navegador
- Detectar sensors inactius



```
except: return ok({})

@app.route("/api/all")
def all_data():
    alerts_data = []
    path = os.path.join(DATA_DIR, "alerts.json")
    if os.path.exists(path):
        try:
            with open(path) as f: alerts_data = json.load(f)[-10:]
        except: pass
    return ok({
        "gps": read("gps_sensor.json"),
        "fuel": read("fuel_sensor.json"),
        "engine": read("engine_temp_sensor.json"),
        "ais": read("ais_sensor.json"),
        "water": read("water_sensor.json"),
        "camera": read("camera_node.json"),
        "alerts": alerts_data,
        "timestamp": time.time(),
        "time_str": time.strftime("%H:%M:%S"),
    })

if __name__ == "__main__":
    print("=" * 45)
    print(" NAVISAFE - API Bridge :5000")
    print(" http://0.0.0.0:5000/api/all")
    print("=" * 45)
    app.run(host="0.0.0.0", port=5000, debug=False)
```

## 8.5 Backend PHP

La part PHP del projecte s'ha utilitzat per implementar el sistema d'autenticació del dashboard.

Els fitxers amb PHP són:

- login.php
- db.php

Aquest sistema permet:

- Validar usuaris
- Iniciar sessió
- Tancar sessió
- Gestionar sessions PHP
- Protegir l'accés al dashboard



```
$db->exec("CREATE TABLE IF NOT EXISTS users (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    username      TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL
)");

$usuaris = [
    ['admin', 'navisafe'],
    ['guillem', '123'],
    ['miquel', '123'],
];
```

## 8.6 Estructura del repositori

```
C:\USERS\GUILLEM TARRADAS\DOCUMENTS\NAVISAFE
├── bridge
│   └── bridge.py
├── docs
│   ├── Memòria Tècnica Navisafe.pdf
│   └── Presentació Navisafe.pdf
├── php
│   ├── db.php
│   └── login.php
├── ros2_ws
│   └── sensors_demo.py
└── web
    ├── dashboard.html
    ├── fons_Navisafe.png
    ├── index.html
    └── logo_Navisafe.png

PS C:\Users\Guillem Tarradas\Documents\Navisafe>
```

# 9. Desplegament

Aquest apartat descriu com s'han posat en funcionament els components principals del sistema Navisafe.

## 9.1 Entorn de producció

El sistema s'ha desplegat sobre una Raspberry Pi 4 amb 4 GB de RAM i Ubuntu Server 22.04 LTS. La Raspberry Pi actua com a servidor local i és accessible des de la mateixa xarxa Wifi o Ethernet del vaixell.

### Especificació simplificada

| Component         | Detall                                |
|-------------------|---------------------------------------|
| Hardware          | Raspberry Pi 4                        |
| Sistema operatiu  | Ubuntu Server 22.04                   |
| Entorn de treball | ROS 2 Humble                          |
| Backend           | Python                                |
| Servidor web      | Apache HTTP Server + PHP              |
| Base de dades     | SQLite                                |
| Xarxa             | Wi-Fi / Ethernet                      |
| Accés remot       | SSH amb clau pública (+ VPN opcional) |

## 9.2 Passos de desplegament

### Pas 1 - Preparació de la Raspberry Pi

S'ha instal·lat Ubuntu Server a la targeta MicroSD mitjançant Raspberry Pi Imager. Durant la configuració inicial s'han definit les credencials d'usuari, el nom de l'equip i l'accés SSH.

### Pas 2 - Instal·lació de ROS2 Humble

S'ha afegit el repositori oficial de ROS2 i s'ha instal·lat la distribució ROS2 Humble per Ubuntu i s'ha verificat el correcte funcionament de ROS2 mitjançant comandes de prova.

### **Pas 3 - Desplegament dels nodes Python**

Els nodes ROS2 desenvolupats en Python s'han transferit al directori del projecte dins la Raspberry Pi.

### **Pas 4 - Configuració d'Apache2 i PHP**

S'ha instal·lat Apache2 juntament amb PHP i el suport per SQLite.

### **Pas 5 - Configuració del bridge Python**

El fitxer bridge.py s'ha configurat com a intermediari entre el panell de control web i els nodes ROS2. Aquest permet enviar i rebre dades en temps real entre la interfície web i el sistema robòtic

## **9.3 Incidències trobades i solucions**

Durant el desplegament van sorgir diverses incidències tècniques. La més destacada va ser la incompatibilitat dels nodes ROS2 escrits en C++ amb l'arquitectura ARM64 de la Raspberry Pi 4. Per trobar una solució, vam investigar projectes reals que feien servir ROS2 en sistemes similars. Ens vam inspirar en Angler, un framework de codi obert per a robots submarins basat en ROS2

([github.com/Robotic-Decision-Making-Lab/angler](https://github.com/Robotic-Decision-Making-Lab/angler)), orientat al control, simulació i coordinació de vehicles submarins. Mirant els seus nodes vam entendre el model de ROS2 i vam poder reescriure els nostres nodes en Python de manera funcional i simple.

| <b>Incidència</b>                                            | <b>Causa</b>                                                          | <b>Solució aplicada</b>                                                                                                                   |
|--------------------------------------------------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Nodes ROS2 C++ no compilaven a la Raspberry Pi               | Incompatibilitat de llibreries amb l'arquitectura ARM64 i ROS2 Humble | Estudi del projecte Angler (robots submarins ROS2) i reescriptura de tots els nodes a Python 3 seguint el model de publishers/subscribers |
| Desconeixement inicial del model de nodes ROS2               | Primera experiència amb ROS2; documentació extensa i complexa         | Anàlisi de projectes reals com Angler per entendre com estructurar nodes de manera simple i funcional                                     |
| Error al panell de control                                   | El bridge bloquejava peticions des del navegador (                    | Addició de la llibreria f i configuració de capçaleres Access-Control                                                                     |
| El panell de cotrol no es visualitzava correctament en mòbil | Disseny inicial no adaptat a pantalles petites                        | Ús de CSS que ajuda d'IA per millorar l'estil responsive                                                                                  |

## 9.4 Validació final del desplegament

Un cop completat el desplegament, s'ha verificat que tots els components funcionen correctament en l'entorn de producció. Les proves s'han realitzat tant des del dispositiu local com de manera remota a través de la xarxa.

- Tots els nodes ROS2 publiquen dades correctament als seus topics
- El bridge API respon en menys de 100ms a totes les crides
- El dashboard és accessible des de qualsevol dispositiu de la xarxa
- L'autenticació PHP funciona correctament amb sessions segures
- El mapa Leaflet.js visualitza la ruta GPS simulada sense errors
- Tots els serveis systemd s'inicien automàticament en reiniciar el sistema

# 10. Resultats i validació

Aquest apartat presenta els objectius aconseguits durant el desenvolupament del projecte Navisafe, les proves realitzades per validar el funcionament del sistema i les evidències obtingudes.

## 10.1 Objectius aconseguits

| Funcionalitat                      | Estat       | Observació                                |
|------------------------------------|-------------|-------------------------------------------|
| CU-01 Monitoratge en temps real    | ✓ Completat | Sensors simulats funcionant amb ROS2      |
| CU-02 Planificació de rutes        | ✓ Completat | Mapa interactiu amb Leaflet.js            |
| CU-03 Dades de navegació i gràfics | ✓ Completat | Dashboard amb visualització en temps real |
| CU-04 Revisió prèvia al viatge     | ✓ Completat | Test automàtic de tots els sensors        |
| CU-05 Gestió de rutes al cloud     | ✓ Completat | APIs PHP amb SQLite funcionals            |
| Autenticació d'usuaris             | ✓ Completat | Login PHP amb sessions segures            |
| Alertes automàtiques               | ✓ Completat | Detecció d'errors i notificacions actives |
| Landing page pública               | ✓ Completat | Web responsive i accessible               |

## 10.2 Proves realitzades

| Component                 | Prova realitzada                            | Resultat    | Observacions                      |
|---------------------------|---------------------------------------------|-------------|-----------------------------------|
| Sensors simulats ROS2     | Publicació de tòpics en temps real          | ✓ Completat | Dades actualitzades cada 2s       |
| Bridge Python             | Crida a /api/fuel, /api/motor               | ✓ Completat | Latència < 100ms                  |
| Panell de control HTML/JS | Visualització en Chrome i Firefox           | ✓ Completat | Responsive en mòbil               |
| Login PHP                 | Accés amb credencials correctes/incorrectes | ✓ Completat | Sessions gestionades correctament |
| Mapa Leaflet.js           | Visualització de ruta GPS simulada          | ✓ Completat | Marcadors visibles                |
| Alertes automàtiques      | Simulació sensor fora de servei             | ✓ Completat | Alerta en 3 errors consecutius    |

## 10.3 Evidències de funcionament

- Captures de pantalla del dashboard amb dades de sensors en temps real
- Logs de ROS2 mostrant la publicació de tòpics cada 2 segons
- Registres del servidor Apache amb les peticions al backend PHP
- Vídeo demostratiu de la revisió prèvia al viatge (CU-04)
- Export de rutes en format JSON des de la interfície web
- Comprovació del funcionament del sistema des de dispositius mòbils

## 10.4 Feedback rebut

El professor Francesc ha tingut un paper important durant el projecte. Ha ajudat a definir i organitzar la idea inicial, així com a orientar el guió de la presentació i el disseny de la interfície web. També va recomanar l'ús de ROS2 com a base del sistema, cosa que ha condicionat l'arquitectura general de Navisafe i ha permès treballar amb tecnologia utilitzada en robòtica real.

# 11. Conclusions i aprenentatges

## 11.1 Aprenentatges tècnics

El desenvolupament de Navisafe ha representat un repte tècnic ja que ha permès als membres de l'equip adquirir i consolidar coneixements en àrees molt diverses de la informàtica i les xarxes.

### **ROS2 i sistemes de temps real**

Un dels principals aprenentatges ha estat l'ús de ROS2, entenent el sistema de comunicació entre nodes amb publicadors i subscriptors, així com el treball amb sistemes distribuïts. També hem aplicat Python de manera pràctica dins del projecte.

### **Integració full-stack**

El projecte ha requerit treballar en totes les capes d'un sistema informàtic: des de la capa física (sensors, Raspberry Pi) fins a la capa d'aplicació (dashboard web), passant per la capa de comunicacions (Wifi, HTTPS) i la capa de serveis (ROS2, Apache, PHP). Aquesta experiència ens ha donat una visió global del cicle de vida d'un sistema tecnològic real.

### **Seguretat informàtica aplicada**

Hem implementat mesures de seguretat reals en el sistema: autenticació amb sessions PHP, comunicació HTTPS amb certificat autosignat, control d'accés al dashboard i registre d'activitat per auditoria.

### **Aprenentatges en equip**

- Ús de Git i GitHub per a la gestió del codi i la col·laboració en equip
- Planificació i seguiment de tasques amb diagrames de Gantt
- Resolució conjunta de problemes tècnics complexos
- Documentació tècnica clara i estructurada per facilitar la comprensió del sistema

## 11.3 Dificultats trobades

| Dificultat                                      | Com s'ha resolt                                                                                             |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Incompatibilitat de ROS2 C++ amb Raspberry Pi 4 | Estudi del projecte Angler i conversió dels nodes a Python 3 seguint el seu model de publishers/subscribers |
| Comunicació entre ROS2 i el dashboard web       | Creació del fitxer <code>bridge.py</code> com a capa intermèdia amb Flask API REST                          |
| Gestió de la concurrència a SQLite              | Implementació del mode WAL i cues d'escriptura per evitar bloquejos                                         |
| Disseny responsive del dashboard                | Ús de CSS Grid i Media Queries per adaptar la interfície a mòbils i tauletes                                |



# 12. Annexos

## 12.1 Fragment de codi

### Backend PHP

```
Archivo  Editar  Ver  H1  i  B  I  S  G  田  v  A
if (isset($body['action']) && $body['action'] === 'logout') {
    session_destroy();
    echo json_encode(['ok' => true]);
    exit;
}

// --- LOGIN ---
$username = trim($body['username'] ?? '');
$password = $body['password'] ?? '';

if ($username === '' || $password === '') {
    echo json_encode(['ok' => false, 'error' => 'Omple tots els camps']);
    exit;
}

$db = getDB();
$stmt = $db->prepare("SELECT * FROM users WHERE username = ?");
$stmt->execute([$username]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);

if ($user && password_verify($password, $user['password_hash'])) {
    session_regenerate_id(true);
    $_SESSION['username'] = $user['username'];
    echo json_encode(['ok' => true, 'username' => $user['username']]);
} else {
    echo json_encode(['ok' => false, 'error' => 'Usuari o contrasenya incorrectes']);
}

```

### Sistema d'autenticació

```
$db = new PDO('sqlite:' . __DIR__ . '/../data/navisafe.db');
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

// Creem la taula si no existeix
$db->exec("CREATE TABLE IF NOT EXISTS users (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    username      TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL
)");

// Afegim els usuaris per defecte si no existeixen
$usuaris = [
    ['admin', 'navisafe'],
    ['guillem', '123'],
    ['miquel', '123'],
];

foreach ($usuaris as [$nom, $contrasenya]) {
    $check = $db->prepare("SELECT id FROM users WHERE username = ?");
    $check->execute([$nom]);
    if (!$check->fetch()) {
        $hash = password_hash($contrasenya, PASSWORD_BCRYPT);
        $ins = $db->prepare("INSERT INTO users (username, password_hash) VALUES (?, ?)");
        $ins->execute([$nom, $hash]);
    }
}

```

## bridge

```
@app.route("/api/status")
def status():
    return ok({"status": "OK", "service": "Navisafe Bridge",
              "timestamp": time.time(), "time_str": time.strftime("%H:%M:%S")})

@app.route("/api/gps")
def gps():    return ok(read("gps_sensor.json"))

@app.route("/api/fuel")
def fuel():   return ok(read("fuel_sensor.json"))

@app.route("/api/engine")
def engine(): return ok(read("engine_temp_sensor.json"))

@app.route("/api/ais")
```

## Node ROS2 Python

```
sensors_demo.py X
C:\Users\Guillem Tarradas\Documents\Navisafe\ros2_ws> sensors_demo.py
20 def simular():
21
22     # Limits zona marítima
23     MIN_LAT = 41.84775 # zona submarinisme Palamós
24     MAX_LAT = 41.84782 # Torre Valentina
25     MIN_LON = 3.13490 # zona submarinisme Palamós
26     MAX_LON = 3.13505 # Torre Valentina
27
28     # Moviment petit aleatori
29     lat += random.uniform(-0.00002, 0.00002)
30     lon += random.uniform(-0.00002, 0.00002)
31
32     # Limitar dins la zona
33     lat = round(max(MIN_LAT, min(MAX_LAT, lat)), 6)
34     lon = round(max(MIN_LON, min(MAX_LON, lon)), 6)
35
36     trail.append({"lat": lat, "lon": lon})
37     trail[:] = trail[-15:]
38
39     fuel = max(0, round(fuel - random.uniform(0, 0.2), 1))
40     temp = max(60, min(round(temp + random.uniform(-1.5, 2.5), 1), 115))
41     sent = max(0, min(round(sent + random.uniform(-1.0, 2.0), 1), 100))
42     vel = round(random.uniform(0.5, 8.0), 1)
43     rumb = round((angle + 90) % 360)
44     n_ais = random.randint(0, 4)
45     vessels = random.sample(VESSELS, n_ais)
46     cam_ok = random.random() > 0.1
47
48     alertes = []
49     ts = time.strftime("%H:%M:%S")
50     if fuel < 20:
51         alertes.append({"nivell": "WARNING", "msg": f"🔥 Combustible baix: {fuel}%", "hora": ts})
52     if temp > 100:
53         alertes.append({"nivell": "CRITICAL", "msg": f"🌡️ Temperatura motor: {temp}°C", "hora": ts})
54     if sent > 25:
55         alertes.append({"nivell": "CRITICAL", "msg": f"💧 Bomba sentina: {sent}%", "hora": ts})
56     if fuel < 10 and alertes:
57         alertes[0]["nivell"] = "CRITICAL"
58
59     dades = {
60         "gps": {"lat": lat, "lon": lon, "trail": trail, "velocitat": vel, "rumb": rumb},
61         "fuel": {"valor": fuel, "estat": "CRITICAL" if fuel < 10 else "WARNING" if fuel < 20 else "OK"},
62         "motor": {"valor": temp, "estat": "CRITICAL" if temp > 100 else "WARNING" if temp > 90 else "OK"},
63         "sentina": {"valor": sent, "estat": "CRITICAL" if sent > 25 else "WARNING" if sent > 15 else "OK"},
64         "ais": {"vaixells": n_ais, "llista": vessels},
65         "camera": {"ok": cam_ok},
66         "alertes": alertes,
67         "hora": ts
68     }
```

## index i dashboard

```
Archivo  Editar  Ver
Δ Sensors desconnectats — executa: <code>python3 main.py --demo</code>
</div>

<!-- KPIs -->
<div class="krow">
  <div class="kpi">
    <div class="kl"> 🔥 Combustible</div>
    <div><span class="kv" id="vf">--</span><span class="ku">%</span></div>
    <div class="kb"><div class="kbf" id="bf"></div></div>
    <span class="bs bok" id="sf">--</span>
  </div>
  <div class="kpi">
    <div class="kl"> 🏎 Motor</div>
    <div><span class="kv" id="vt">--</span><span class="ku">°C</span></div>
    <div class="kb"><div class="kbf" id="bt"></div></div>
    <span class="bs bok" id="st">--</span>
  </div>
  <div class="kpi">
    <div class="kl"> 💧 Sentina</div>
    <div><span class="kv" id="vs">--</span><span class="ku">%</span></div>
    <div class="kb"><div class="kbf" id="bse"></div></div>
    <span class="bs bok" id="sse">--</span>
  </div>
  <div class="kpi">
    <div class="kl"> 📶 AIS</div>
    <div><span class="kv" id="va" style="color:var(--blue)">--</span><span class="ku"> vaix.</span></div>
    <div id="ais-d" style="margin-top:.6rem;font-size:.68rem;color:var(--muted);font-family:'DM Mono',monospace">--</div>
  </div>
</div>

Ln 594, Col 35 | 30.639 caràcters. | Texto sin formato | 100% | Windows (CRLF) | UTF-8
```

```
Archivo  Editar  Ver
<a href= "dashboard.html" class= "btn-plan btn-plan-o">Provar ara</a>
</div>
<div class="plan featured">
  <div class="plan-badge">MÉS POPULAR</div>
  <div class="plan-name">Navegant</div>
  <div class="plan-price">19€<span>/mes</span></div>
  <p class="plan-desc">Per a navegants habituals que necessiten control total del seu vaixell.</p>
  <ul>
    <li>Tots els sensors</li>
    <li>Alertes SMS + email</li>
    <li>Historial 90 dies</li>
    <li>3 dispositius</li>
    <li>Càmera en directe</li>
    <li>Revisió prèvia automàtica</li>
  </ul>
  <a href="dashboard.html" class="btn-plan btn-plan-p">Començar</a>
</div>
<div class="plan">
  <div class="plan-name">Professional</div>
  <div class="plan-price">49€<span>/mes</span></div>
  <p class="plan-desc">Per a empreses nàutiques i flotes de vaixells amb necessitats avançades.</p>
  <ul>
    <li>Flota de vaixells</li>
    <li>API pròpia</li>
    <li>Historial il·limitat</li>
    <li>Usuaris il·limitats</li>
    <li>Suport prioritari 24/7</li>
  </ul>
  <a href="dashboard.html" class="btn-plan btn-plan-p">Començar</a>
</div>

Ln 645, Col 14 | 22.695 caràcters. | Texto sin formato | 100% | Windows (CRLF) | UTF-8
```

## 12.2 Coneixements del cicle:

| Mòdul                         | Aplicació al projecte Navisafe                              |
|-------------------------------|-------------------------------------------------------------|
| MP01 · Muntatge i manteniment | Connexió de sensors i perifèrics a la Raspberry Pi 4        |
| MP02 · SSOO monolloc          | Instal·lació i configuració d'Ubuntu Server 22.04           |
| MP04 · SSOO en xarxa          | Gestió de serveis systemd, SSH i accés remot                |
| MP05 · Xarxes locals          | Configuració de xarxa Wi-Fi/Ethernet, IP estàtica, firewall |
| MP06 · Seguretat informàtica  | HTTPS, autenticació, sessions PHP, registre d'auditoria     |
| MP07 · Serveis de xarxa       | Configuració Apache2, DNS local,                            |
| MP08 · Aplicacions web        | Desenvolupament del dashboard HTML/CSS/JS i backend PHP     |
| MP10 · Empresa i emprenedoria | Definició del model de negoci i valor del producte Navisafe |
| MP11 · Anglès tècnic          | Documentació en anglès, vocabulari tècnic, presentació oral |
| MP12 · Síntesi                | Integració de tots els mòduls en un projecte real           |

## 12.3 Wiki

In this project, we worked on the four main skills: listening, speaking, reading, and writing. In speaking, we explained different topics in class, hardware setup, web design, and ethical hacking. Marina corrected us while we were speaking, which helped us improve our pronunciation and fluency.

For listening, we paid attention to the videos that Marina puts in class and her explanations. This helped us understand technical vocabulary and how to use it correctly in context.

In writing, we worked on organizing information and writing clear explanations about the topics. We also corrected our mistakes after the feedback from Marina.

In reading, we improved our understanding by reading texts about the topics. This helped us learn new vocabulary and better understand concepts like computer components, web technologies, and cybersecurity.

We also learned and used specific vocabulary, such as CPU, RAM, motherboard, web design, responsive websites, cybercrime, and ethical hacking. Using this vocabulary correctly helped us communicate more professionally. This project helped us improve our English level, especially in technical language, and we practiced all the skills.

### Topics:

#### 1. Hardware Setup & Safety

##### 1. MACRO-TOPIC: Hardware Setup & Safety

Hardware refers to the physical parts of a computer that you can see and touch. This includes internal components like the CPU, RAM, motherboard, storage devices, GPU, power supply, and cooling systems, as well as external components such as monitors, keyboards, mice, printers, and speakers. Each part has a specific function, and knowing these functions is important to build, maintain, and use a computer correctly.

**The main internal components include:**

- CPU (Central Processing Unit): The “brain” of the computer that processes instructions and performs calculations.
- RAM (Random Access Memory): Temporary memory that stores data for programs that are currently running.
- Motherboard: The main circuit board that connects all components and allows them to communicate.

- **Storage Devices:** HDDs (Hard Disk Drives) and SSDs (Solid State Drives) store programs, files, and the operating system. SSDs are faster and more reliable than HDDs.
- **GPU (Graphics Processing Unit):** Handles graphics and images, important for games, design, and video editing.
- **Power Supply Unit (PSU):** Provides electricity to all components safely.
- **Cooling Systems:** Fans or liquid cooling prevent components like the CPU and GPU from overheating.

A proper hardware setup means carefully assembling and connecting all internal and external components. This includes installing the CPU with thermal paste, connecting RAM and storage devices, managing cables for good airflow, and ensuring all parts fit correctly on the motherboard. Safety is very important, so using anti-static wrist straps, handling sensitive components carefully, and following safety rules can prevent damage or accidents.

Computer security is also an essential part of hardware setup. It protects the computer and its data from unauthorized access, damage, or theft. Security tools include firewalls, antivirus software, and encryption, while security measures include using strong passwords, keeping software updated, and physically protecting the hardware. Combining correct hardware setup with proper security practices ensures computers operate safely, efficiently, and reliably.

## **10. Web Apps & CMS**

### **MACRO-TOPIC 10: WEB DESIGN & WEB TECHNOLOGY**

This topic explores the creation, functionality, and evolution of websites, covering web design principles, types of websites, development, tools, and modern trends including Web 2.0 interactivity.

#### **What is Web Design?**

Web design is the process of creating the visual appearance and user experience of a website or web application. It includes layout, colors, typography, images, navigation, and overall branding. Web design ensures that websites are both attractive and user-friendly, especially on mobile devices.

#### **History of Web Design**

The first website was created by Tim Berners-Lee in 1991, initially for sharing information. Early websites were simple and text-based. Key milestones include:

- 1995 – JavaScript introduced
- 1998 – CSS for styling websites

- 2000s – Content Management Systems like WordPress
- Recent years – Responsive design for mobile devices

Modern web design focuses on aesthetics, interactivity, and user experience.

### **Web Design vs Web Development**

- Web Designer → creates the look, layout, branding, and usability of a website
- Web Developer → implements the functionality using HTML, CSS, JavaScript, and server-side languages like PHP or Python

Designers and developers often work together, though some individuals handle both roles.

### **Web Design Principles**

Key principles guide effective web design:

- Balance – evenly distribute text, images, and whitespace
- Contrast – highlight important elements with color, size, or style
- Hierarchy – structure content for easy navigation and comprehension
- Repetition – maintain consistency in colors, fonts, and layout
- User-centric – design based on user needs and ease of use

### **Types of Web Design**

- Static Websites – simple HTML/CSS, fixed content, mostly informational
- Single-Page Websites – all content on one page, often for portfolios or small apps
- Dynamic Websites – interactive sites with changing content, require programming
- Responsive Websites – adapt layout to different devices and screen sizes

### **Web Design Tools**

- Complete Web Platforms – create websites without coding: Wix, Squarespace
- Layout & Design Tools – professional design software: Adobe Photoshop, Sketch
- Mockup & Wireframing Tools – prototype websites: Figma, Balsamiq

### **Web Technology**

Web technology is the communication and coding infrastructure that allows websites and apps to function. It relies on markup languages (HTML), styling (CSS), and scripting (JavaScript). Modern web technology also supports databases, multimedia, apps, and interactive services.

## **Web 2.0 Overview**

Web 2.0 emphasizes interactivity, social sharing, and online collaboration, unlike Web 1.0 which was mostly read-only. Key features include:

- Connectivity – people-to-people, users-to-software, and online services
- Tools – web-based email, presentation tools, video sharing, and mobile apps
- Applications – web apps for communication, writing, data analysis, and organization

Web 2.0 allows users to publish content, interact, and perform tasks online, making the web more dynamic and user-centered.

## **Careers in Web Design**

- Graphic Designer – visual aesthetics, layouts, branding
- UX Designer – user experience, navigation, and usability
- UI Designer – interactive elements like buttons, icons, and menus

Web design careers often involve collaboration with developers and can range from independent projects to large-scale corporate websites.

## **15. Ethical Hacking & Security**

### **MACRO-TOPIC 15: ETHICAL HACKING & SECURITY – EXTENDED SUMMARY**

This topic explores the role of cybersecurity in modern society, focusing on cybercrime, computer hacking, ethical hacking, and the importance of protecting personal and organizational data in an increasingly digital world.

### **Internet Use and Digital Responsibility**

Most people spend several hours a day online using services such as social media, online banking, cloud platforms, email, and gaming services. While these tools provide convenience and connectivity, they also expose users to cyber threats. Using the same password across multiple accounts, clicking on suspicious links, or sharing personal information increases the risk of cyberattacks. Developing safe online habits is a key part of digital responsibility.

### **Cybercrime: Definition, History, and Types**

Cybercrime refers to criminal activities carried out using computers or digital networks. It began with early hacking attempts, where individuals accessed computer systems illegally, often out of curiosity or for technical challenges. Over time, cybercrime evolved



into more organized and harmful activities aimed at financial gain, data theft, and large-scale disruption.

Common types of cybercrime include phishing scams, identity theft, credit card fraud, data breaches, software piracy, online fraud, and cyberterrorism. These crimes can cause financial losses, emotional distress, and serious security risks for individuals, businesses, and governments.

To address these threats, governments have established cybercrime laws. In the United States, the Computer Fraud and Abuse Act (CFAA) criminalizes unauthorized access to computer systems and networks. Penalties vary depending on the offense and can range from fines to long prison sentences.

### **Computer Hacking and Its Evolution**

Computer hacking is the act of accessing computer systems without permission in order to view, modify, or destroy data. Although hacking is often associated with criminal behavior, it originally had a more positive meaning. In the 1960s and 1970s, hackers were skilled individuals who modified systems to improve performance and efficiency.

As computers became more widespread in the 1980s, hacking increasingly involved illegal activities. This led to the distinction between malicious hackers (black hats), who seek to cause harm, and ethical hackers (white hats), who work to improve system security.

### **Preventing Cyber Attacks**

Preventing hacking and cybercrime requires a combination of technical measures and user awareness. Important prevention strategies include regularly updating software and operating systems, using strong and unique passwords, avoiding unsecured wireless networks, verifying the authenticity of websites and emails, protecting against phishing attempts, logging out of online sessions, and using up-to-date antivirus software.

### **Ethical Hacking and Its Purpose**

Ethical hacking involves authorized efforts to identify and fix security weaknesses before cybercriminals can exploit them. Ethical hackers simulate real attacks to test networks, applications, hardware, and even human behavior through social engineering techniques. Many organizations rely on ethical hacking as a proactive way to strengthen their cybersecurity defenses.

## **Information Gathering in Ethical Hacking:**

Information gathering is the first stage of ethical hacking and is considered a pre-attack phase. During this process, ethical hackers collect publicly available information, identify network ranges, detect active machines, scan for open ports, determine operating systems and services, and create detailed maps of network structures. This helps organizations understand their vulnerabilities and improve their security posture.